LANGUAGE IS A DIRECTION IN EMBEDDING (LAYER 0) SPACE

Plinio Avendaño
Department of Computer Science
UTEC
plinio.avendano@utec.edu.pe

ABSTRACT

The linear representation hypothesis (LRH) states that concepts are encoded as linear subspaces of a model's representation space. This means that there exist directions that represent concepts such as gender, sentiment, or refusal. This was first observed in word embeddings, where simple arithmetic revealed a direction for gender that could be added to another embedding to steer its meaning (the famous *king* − *man* + *woman* ≈ *queen*). In LLMs, the LRH has been studied for the model's internal and output representations and has motivated lines of research such as superposition and activation steering. However, almost all of this work operates on hidden layers, and whether the input embedding space of an LLM still contains directions that the model reads as properties of its input remains largely unexplored. In this work, we find through gradient-based optimization a single vector that, when added identically to every token embedding of the user's question, makes the model answer in French in 100% (Llama-3.2-3B-Instruct) and 97% (Qwen3-4B-Instruct-2507) of held-out English questions, with the same accuracy as when the question is written in French, while every token remains closest to its original embedding. Analogous vectors for Spanish and German reach 96–99%, and the effect transfers to input languages never seen in training (Italian and Portuguese). Moreover, the effect is signed and largely language-specific. Subtracting a language's vector from questions in that language moves the answer out of it. In Llama, only 3–17% of the answers remain in that language. In Qwen, 7% remain for German, and the effect is weaker for French and Spanish (52–62%). Vectors of other languages and random vectors leave most answers unchanged. Position-bound vectors trained on the same task cannot be inverted. Behavioral tests show that the vector changes the model's perception of the input rather than acting as an instruction. Asked which language a patched English question is written in, the model names the language of the vector in 76–99% of cases, while a soft prompt trained on the same objective acts as an instruction to answer in that language. These results show that the language of the input is linearly accessible at the embedding layer, before any computation by the network, bringing the LRH back to the embedding space where it began.

1 Introduction

The linear representation hypothesis (LRH) states that high-level concepts are represented as linear directions, or more generally linear subspaces, in a model's representation space [Park et al., 2024]. It started as an observation from Mikolov et al. [2013], who found that word embeddings encode semantic and syntactic relationships that simple arithmetic can recover ($king - man + woman \approx queen$). This is interesting because it implies that $king - queen \approx man - woman$; that is, there is a direction for gender. Remarkably, this happens without being the explicit objective of the algorithm, learning a dimensional mapping from words to vectors yields this kind of emergent relationship.

In LLMs the hypothesis has been extended to the hidden states of the model (activations/attention heads), and it has shaped the field in two ways. First, by opening new lines of research, and second, by enabling practical methods to control model behavior.

Works such as Toy Models of Superposition by Elhage et al. [2022], where they find that networks represent more concepts than they have neurons by storing them in 'superposition', build on this. Sparse autoencoders [Bricken et al., 2023, Templeton et al., 2024] take this one step further, decomposing these superposed activations into interpretable

features. On the other hand, activation steering is a method to control model behavior that relies on linear directions. It works by subtracting the activations of contrastive pairs of prompts in middle to top layers, which yields directions that can enhance or suppress behaviors. Using this method, Arditi et al. [2024] showed that refusal is mediated by a single direction in the residual stream, and Ball et al. [2024] showed that different jailbreaks also share a common direction.

However, almost all of this work, both on reading features and on writing them, operates on the hidden layers of the model. The input embedding space, where the LRH was born, has received much less attention in LLMs: Dar et al. [2023] use it to interpret the parameters of the model, and Freenor and Alvarez [2025] argue that modern text representations have lost the simple vector arithmetic of early word embeddings and in consequence the authors model semantic transformations as rotations in the output space of embedding models. Whether the input embedding space of an LLM still contains linear directions that the model reads as properties of its input remains largely unexplored. We intervene in this space, hypothesizing that if adding a single vector to every token changes what the model perceives about its input, that property is linearly accessible at layer 0.

In this work, we show that language is a direction that, when added to the token embeddings of the user's prompt, changes the language of the model's answer to French, both from English and from other input languages. This is achieved by using gradient descent to optimize a single vector, with the objective of minimizing the cross-entropy of French target answers under teacher forcing, over a collection of questions in English, Spanish, and German. Moreover, applying the same procedure, we also find analogous directions for Spanish and German, and we reproduce all results in two models of different families. Our contributions are as follows:

- We show that the input embedding space of an LLM contains linear directions that the model reads as the language of its input. In Llama-3.2-3B-Instruct and Qwen3-4B-Instruct-2507, adding a single vector to every token of an English question makes the model answer in French in 100% and 97% of held-out questions (respectively) and analogous vectors for Spanish and German reach 96–99%. The model retains the same accuracy as when the question is written in the target language, while every token remains closest to its original embedding. The effect transfers to input languages never seen in training, with 83–100% of Italian and Portuguese questions answered in the target language.

- We show that the effect reverses with the sign of the vector and is largely language-specific. Subtracting a language's vector from questions written in that language moves the answer out of it. In Llama, only 3–17% of the answers remain in the language of the question. In Qwen, this holds for German (7%), while for French and Spanish the effect is partial (52–62% remain). Subtracting another language's vector leaves at least 80% of the answers in the original language (except between French and German in Llama) and random vectors of the same norm leave 92–100%. In contrast, position-bound patches trained on the same task cannot be inverted, even at twice their magnitude.

- We introduce behavioral tests that distinguish a change in the perceived input from an instruction, and show that the direction acts on perception while a canonical soft prompt trained on the same objective acts as an instruction. Asked which language a patched English question is written in, the model names the language of the patch in 76–90% of cases in Llama and 88–99% in Qwen (0–2% for a random vector), while answering in English. Under an explicit instruction to answer in English, a patched question behaves closely like a question genuinely written in the target language, while a soft prompt trained on the same objective overrides the instruction in 97–100% of cases.

- We discuss the limits of this view. The directions of different languages share a common component, which makes subtraction leak between French and German in Llama. The inversion is only partial in Qwen for French and Spanish, where the answers that leave the language go mostly to Romanian. Finally, changing the perceived language is not unique to the direction. Three distinct vectors added to the first tokens of the question also change it, although less consistently in Llama, but unlike the direction they cannot be inverted. This separates two properties, since acting on the question tokens is what changes perception, while sharing a single vector across all of them is what makes it a direction.

2 Background

We now introduce the definitions necessary for this work.

2.1 Transformer notation

We consider a decoder-only transformer with L layers and hidden dimension d. Given an input sequence of tokens $x_1, \ldots, x_T$ from a vocabulary $\mathcal{V}$, each token is mapped to a vector by the embedding matrix $W_E \in \mathbb{R}^{|\mathcal{V}| \times d}$:

$$e_i = W_E[x_i] \in \mathbb{R}^d. \tag{1}$$

These token embeddings form the initial state of the residual stream, which we refer to as layer 0:

$$h_i^0 = e_i. \tag{2}$$

Each layer ℓ then reads from the residual stream and adds its output back to it. In both models we study, Llama-3.2 [Touvron et al., 2023, Grattafiori et al., 2024] and Qwen3 [Yang et al., 2025], each layer consists of an attention block and an MLP block, both preceded by an RMSNorm [Zhang and Sennrich, 2019]:

$$\tilde{h}_i^\ell = h_i^\ell + \mathrm{Attn}^\ell\big(\mathrm{RMSNorm}(h^\ell)\big)_i, \tag{3}$$

$$h_i^{\ell+1} = \tilde{h}_i^\ell + \mathrm{MLP}^\ell\big(\mathrm{RMSNorm}(\tilde{h}_i^\ell)\big), \tag{4}$$

where $\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_j x_j^2 + \epsilon}} \odot g$, with a learned gain $g \in \mathbb{R}^d$. The attention block is the only component that moves information between positions: position i can only attend to positions $j \leq i$. After the last layer, the next-token logits are obtained as

$$\mathrm{logits}_i = W_U \,\mathrm{RMSNorm}(h_i^L), \tag{5}$$

where $W_U \in \mathbb{R}^{|\mathcal{V}| \times d}$ is the unembedding matrix. For the two models, the embedding and unembedding matrices are tied, that is, $W_U = W_E$.

Two properties of this architecture are relevant to our work. First, since every layer adds to the residual stream rather than overwriting it, a vector added to h_i^0 remains present in all subsequent layers, unless later layers write in the opposite direction. Second, every block reads the residual stream through an RMSNorm, so what a layer sees depends on the direction of h_i^ℓ relative to its norm, not only on its absolute value.

2.2 Concepts or features

Elhage et al. [2022] provide three working definitions for features:

- Features as arbitrary functions: Any function of the input.
- Features as interpretable properties: "The presence of human understandable ‘concepts’ in the input". Such as gender, sentiment, etc.
- Neurons in sufficiently large models: Properties of the input to which a sufficiently large model would dedicate a neuron.

We define language (such as French) as a concept, so naturally we will stick with definition number two and will use concept and feature as analogous words. Nevertheless, it is important to note that the authors also acknowledge that this definition may not be enough, because models may represent features that we humans do not understand. However, because in this work we are dealing with human concepts this definition suffices.

They also hypothesize that features are represented as directions in *activation space*, even though the features themselves are nonlinear functions of the input. That is, a feature such as the language of a text is computed by the network from its tokens, and only after passing through sufficient layers the concept is represented linearly. This makes the input embeddings, where no such computation has happened yet, an unlikely place for it.

Yet their main example of features as directions comes from embeddings, the gender direction of Mikolov et al. [2013]. Therefore, the input embeddings of an LLM are also a natural place to look for this kind of structures. In a transformer, the embedding matrix is not a fixed lookup table, it is learned jointly with the rest of the model during pretraining [Vaswani et al., 2017], so it is shaped by the same training pressure as the hidden layers. For this reason, we hypothesize that it may already contain directions that the model can read as concepts. This is the space we study in this work.

2.3 Soft prompts

Adapting a pretrained model to a new task usually requires either fine-tuning its weights or writing a text prompt that describes the task. Soft prompts offer a middle ground, instead of words, the prompt is made of continuous vectors that are learned directly from data, while the model's weights stay frozen.

Prompt tuning [Lester et al., 2021] implements this by prepending k trainable vectors $p_1, \ldots, p_k \in \mathbb{R}^d$ to the token embeddings of the input:

$$[p_1, \ldots, p_k, e_1, \ldots, e_T]. \tag{6}$$

These vectors live in the same space as the token embeddings, but they do not need to correspond to any token in the vocabulary. They are optimized by gradient descent on the task loss, so that the frozen model, reading them as if they were part of the prompt, performs the desired task. Beyond task adaptation, the same idea has been used to find universal jailbreaks [Schwinn et al., 2024] and to compress long texts into a single input vector [Kuratov et al., 2025].

Soft prompts are relevant to this work because they also optimize vectors in the embedding space of a frozen model. The crucial difference is that, even when they are added to the prompt tokens (like in the setup of Schwinn et al. [2023]) they are bound to the positions where they were optimized. Nothing in their training forces them to represent a direction or a transformation of the prompt; rather, they behave as a compressed instruction [Mu et al., 2023] that is, a prompt written in vectors rather than tokens. In contrast, the direction we study neither adds nor depends on positions. A single vector is added identically to the embedding of every token of every question.

This difference leads to two predictions that separate a direction from a soft prompt. The first concerns how the model reads the vector. A soft prompt acts as an instruction, so the model should follow it without changing what it perceives about the input. A direction that encodes the language of the input should instead change that perception, so that the model reports an English question as written in French. The second concerns invertibility. A direction has an opposite, so subtracting it should remove the property it encodes. Arithmetic on soft prompts has been explored, but as interpolation between the parameters of trained prompts, used to combine classification tasks or to initialize new ones [Belanec et al., 2024]. This is not the representation-level arithmetic of word embeddings [Mikolov et al., 2013] or function vectors [Todd et al., 2024], and to our knowledge, inverting a behavior by negating a soft prompt has not been shown. We test both predictions in Section 4, comparing our direction with position-bound patches trained on the same objective.

3 Method

3.1 The patch (direction)

We use Llama-3.2-3B-Instruct ($L = 28$, $d = 3072$, $|\mathcal{V}| = 128{,}256$) and Qwen3-4B-Instruct-2507 ($L = 36$, $d = 2560$, $|\mathcal{V}| = 151{,}936$). Inputs follow each model's chat template, with a system block, a user block containing the question, and the header of the assistant turn. Let $\mathcal{Q}$ be the set of positions of the question tokens in the user block. The patch is a single vector $v \in \mathbb{R}^d$ that is added as follows:

$$e'_i = \begin{cases} e_i + \alpha v & \text{if } i \in \mathcal{Q}, \\ e_i & \text{otherwise,} \end{cases} \tag{7}$$

where α is a scalar coefficient. We use $\alpha = 1$ to add the direction and $\alpha = -1$ to subtract it. The system prompt, the role headers, and special tokens such as <|eot_id|> in Llama or <|im_end|> in Qwen are left untouched, and so are the tokens the model generates. The patch modifies only the user section of the prompt.

Because the same vector is added at every position of the question, it cannot depend on where a token appears in the sequence or on which token it is added to (position). This is what distinguishes it from a soft prompt, and it makes v a candidate direction of the embedding space rather than a set of learned tokens. Whether the model reads it as a direction is what the tests of Sections 4.3 and 4.4 evaluate.

To compare our method with soft prompting, we also train, for each language and each model, two patches with position-bound formats and the same objective. This gives three formats:

- **Shared (ours).** A single vector v, added to every question position as in Eq. (7).
- **Question-3.** Three distinct vectors, added to the first three tokens of the question, similar to Schwinn et al. [2023] but shared across inputs.
- **Header-3.** Three distinct vectors, added to the three tokens of the assistant header (`assistant`, <|end_header_id|>, \n\n in Llama, and <|im_start|>, `assistant`, \n in Qwen). Since these positions are the same for every input, this is the canonical equivalent to a soft prompt [Lester et al., 2021].

3.2 Finding the patch (training)

The patch is optimized directly, with the model frozen. For a target language lan, we build a training set of pairs (q, y), where q is a question written in a source language $s \neq$ lan and y is an answer to that question written in lan. Questions in the target language are excluded from training. For French, the source languages are English, Spanish, and German.

Given a pair (q, y), we patch the embeddings of q as in Eq. (7) and compute the cross-entropy of the target answer under teacher forcing, averaged and restricted over its first K tokens:

$$\mathcal{L}(v) = \mathbb{E}_{(q,y)} \left[-\frac{1}{K} \sum_{k=1}^{K} \log p\big(y_k \mid q \oplus v,\ y_{<k}\big) \right] + \lambda \|v\|_2^2, \tag{8}$$

where $q \oplus v$ denotes the prompt with the patch applied to the question tokens, and λ controls an L_2 penalty on the norm of the patch. We use $K = 8$, the first tokens of the answer are where the model commits to a language, and once it does, it maintains it for the rest of the generation. Restricting the loss to them makes the objective about the language of the answer rather than about reproducing a particular answer or style.

We optimize v with sign-SGD, which updates each coordinate by a fixed step in the direction of the sign of its gradient:

$$v \leftarrow v - \eta_t \operatorname{sign}\big(\nabla_v \mathcal{L}\big), \qquad \eta_t = \frac{\eta_0}{2}\left(1 + \cos\frac{\pi t}{T-1}\right), \tag{9}$$

with $\eta_0 = 2.5 \times 10^{-4}$ and a cosine schedule that anneals the step to zero over the T update steps.

Each update uses the average gradient over a batch of 28 training questions, each presented in its three source languages, accumulated with one forward and backward pass per pair. We take 20 update steps on each batch before moving to the next one. We train for 10 epochs of 21 batches, for a total of $T = 4{,}200$ updates, with $\lambda = 0.0915$. The patch is initialized at zero and the order of the batches is fixed, so training is deterministic. At the end of each epoch, we measure the cross-entropy of the first five target tokens on the first 20 training questions, in each of their source languages, and keep the patch of the epoch with the lowest value. No held-out question is used for this selection. The selected epoch is the last one for every patch except two Question-3 patches (Spanish in Llama, epoch 9, and German in Qwen, epoch 8).

The patches for Spanish and German, as well as the Question-3 and Header-3 patches of each language, are trained with the same recipe, using the other two languages plus English as sources.

3.3 Data

Questions. We use a bank of 700 short questions with a verifiable answer, divided into seven categories of 100 questions that differ in the form of the prompt:

- *Factual*: *What is the capital of Iceland?*
- *Factual with other openings*: *Where was Mozart born?*
- *Imperative*: *State the capital of Mongolia.*
- *Explanatory imperative*: *Detail why ice floats on water.*
- *Explanatory question*: *How do geckos climb walls?*
- *Short fragment*: *Capital of Peru?*
- *Conversational*: *Hey, who directed Jaws?*

All questions are at most 25 tokens long. Both the questions and their translations were generated by Claude Opus 5. Each question is translated into French, Spanish, and German, which are the target languages of the patches, and into Italian and Portuguese, which are used only for evaluation and never appear during training. We reviewed every translation by hand against the English original, checking that it preserves both the meaning and the form of the question (an imperative stays an imperative, a fragment stays a fragment, and a conversational question keeps its casual tone). This review corrected 38 translations in 31 questions (mostly mistranslated titles, changes in meaning/form, and grammar errors). The bank is interleaved by category, so splitting it by position keeps it balanced. The first 588 questions are used for training and the last 112 (16 per category) are held out for evaluation. We select questions with a verifiable answer so that we can measure whether the patch induces errors or hallucinations, and we vary their form so that the patch cannot rely on a single question template.

Target answers. The target y for each question is the model's own answer to the question written in the target language, without any instruction:

$$y = M\big(q_{\text{lan}}\big), \tag{10}$$

where lan is the language of the question. Targets are generated with greedy decoding for up to 250 tokens, so that explanatory answers are not cut before the explanation. Each model is trained on its own answers, so Llama-3.2-3B-Instruct and Qwen3-4B-Instruct-2507 generate their targets separately from the same questions. Since the target is what the model answers when the question is genuinely written in the target language, and not what it answers when told to respond in that language, the patch is trained to reproduce an input in that language rather than an instruction.

We reviewed all 2,100 targets of each model by hand. We kept the style of the model and corrected factual errors (e.g., *Jean-Pierre Jeunet* as the director of *Jaws*), words in another language, and filler openings such as *¡Claro!*. This changed 447–488 of the 700 targets per language in Llama and 526–557 in Qwen. It is important to note that most of these changes correct factual errors (350–410 targets in Llama and 414–482 in Qwen), but only a minority of them concern the answer itself: 124–138 targets in Llama and 70–96 in Qwen did not contain the correct answer, while the rest gave the correct answer followed by errors in the explanation (wrong dates/mechanisms or invented names). We also made sure that no target is a bare answer (e.g., *Pretoria* instead of *La capital de Sudáfrica es Pretoria*), since a bare answer is often identical across languages and gives the patch no signal about the language. After the review, every target is in the target language and answers correctly. These corrected targets are used only for training. In evaluation, we compare the patch against the model's answers to the questions written in the target language. These answers are generated again at evaluation time and are not corrected.

Open-ended prompts. To test generalization beyond the question formats seen in training, we use an additional set that never enters training: 99 open-ended prompts (*What's the weather like...*) which differs from the training questions in both form and length. These questions do not have a factual answer, so we do not measure accuracy.

The set is also translated into French, Spanish, and German, which provides the native control on them.

3.4 Evaluation

We report two main metrics:

- **Language rate:** The fraction of responses written in the target language, as identified by the language detector GlotLID [Kargaran et al., 2023] on the full response.
- **Accuracy:** The fraction of responses that answer the question correctly, measured by matching the response against a list of accepted answers for each question, with aliases in every evaluation language (e.g., *Athens*, *Athènes*, *Atenas*, *Athen*).

The language rate measures whether the patch changes the language of the answer, and accuracy measures whether it does so without damaging the model's reading of the question. We compare three conditions on the held-out questions. *None* is the English question without the patch. *Native* is the question written in the target language, it serves as the reference for how often and how accurately the model answers in that language. Its answers are generated at evaluation time and are not corrected. *Patch* is the English question with the patch added.

We report these metrics with the held-out dataset as well as the open set (without accuracy due to being open).

3.5 Behavioral tests and controls

Beyond showing that the patch switches the language of the answer, we test two properties. First, whether its effect reverses with its sign (as expected from a direction). Second, whether it acts on the model's reading of the input or on its decision about the output. All tests use the held-out questions on the three patches plus the position-bound vectors.

Subtraction. For each language lan $\in \{\text{fr}, \text{es}, \text{de}\}$ and in both models, we subtract the patch v_{lan} from questions written in lan ($\alpha = -1$ in Eq. (7)), which the model already answers in lan without any patch, and measure the distribution of response languages and accuracy. A direction should move the answer out of lan, while a soft prompt has no reason to. We compare with three controls. First, we subtract the patches of the other two languages, which share a common component with v_{lan} but encode a different language. If the effect is specific to lan, these should leave the answer in lan. Second, we subtract random vectors with the same norm as v_{lan}, to test whether any displacement of this size is enough to change the language. Third, we subtract the position-bound Question-3 and Header-3 patches of the same language, each at the positions where it was trained, and Question-3 also at twice its magnitude ($\alpha = -2$). If these act as learned tokens rather than as a direction, subtracting them should leave the answer in lan. Each position-bound patch is also paired with a random vector of the same shape, norm, and positions.

Input or instruction. A patch that makes English questions receive French answers admits two readings. It could alter the model's reading of the input, making it perceive the question as written in French, so that the model answers in French by following its usual behavior of replying in the language of the question. Alternatively, it could act on the decision about the output language, as the instruction *Answer in French* would. We argue that the first reading is correct, and we refer to it as a "translation" of the input (a change in the perceived language of the question, not in its tokens). Standard metrics cannot distinguish the two readings, so we design two tests in which the patch is applied only to the span of the question, embedded in a longer message that asks for something else:

- **Language identification.** We ask the model which language the question is written in (*What language is the following question written in? Reply with only the name of the language.*) and record the first language name in the response, as well as the language in which the response is written. If the patch alters the reading of the input, the model should report in English the language of the patch. If it acts as an instruction, the model should report English but write its response in the language of the patch. As we expect the answer to be short (1 word) we classify the language by hand when GlotLID returns unknown.
- **Conflicting instruction.** We append *Please answer in English* to a patched English question, and compare with the same instruction applied to a question genuinely written in the target language. If the patch changes the perceived input, the patched question should behave like the genuine question, in whatever way the model resolves the conflict.

In both tests, the controls are the unpatched question, the question genuinely written in the target language, and, for each patch, a random vector with the same shape, norm, and positions.

These tests also position the patch with respect to the two families of methods discussed in Section 2. A soft prompt adds tokens that act as a compressed instruction, and activation steering writes a behavior into the hidden states of the model; in both cases, the intervention tells the model what to do. The patch, in contrast, modifies the initial representation of the question itself, before any computation by the network. If it acts on the model's reading of the input, the model does not receive an order to answer in French. It receives a question that, in its own representation, is written in French, and answers it as it would answer any French question. The instruction reading would make the patch a continuous counterpart of a soft prompt; the input reading would make it a different kind of intervention, one that edits what the model perceives rather than what it is asked to do. The two tests are designed to tell these two cases apart.

4 Results

All results in this section are computed on the 112 held-out questions of each language, except for the open set.

4.1 A single vector switches the answer language

On English questions, the French patch turns all answers of Llama into French and 97% of those of Qwen, and the Spanish and German patches reach 96–99% in both models (Table 1). These rates match those of the native questions (95–100%), without any instruction being present in the prompt. Accuracy is also preserved. It drops from 97–99% on English questions to 72–84% on native questions, which is the cost of the model answering in another language, and the patch matches this reference. In Qwen, the patch and the native question differ by at most one question, and in Llama the patch is slightly more accurate in all three languages, with most disagreements split in both directions (Won/Lost).

Table 1: Held-out English questions. $\|v\|$: L2 norm of each patch. None, Native, and Patch as defined in Section 3.4. Δ: accuracy of the patch minus accuracy of the native question, computed from unrounded values. *Won/Lost*: questions that the patch answers correctly and the native question does not, and vice versa.

			Target language (%)			Accuracy (%)				
Model	Patch	$\|v\|$	None	Native	Patch	None	Native	Patch	Δ	Won/Lost
Llama-3.2-3B	French	0.88	0	99	100	97	76	79	$+2.7$	6/3
	Spanish	0.91	0	96	99	97	72	82	$+9.8$	18/7
	German	0.94	0	95	99	97	78	81	$+3.6$	13/9
Qwen3-4B	French	0.92	0	100	97	99	84	84	0.0	7/7
	Spanish	0.96	1	97	96	99	84	84	0.0	6/6
	German	1.00	0	98	97	99	84	83	-0.9	5/6

The same holds for held-out questions written in the other two training languages of each patch, where the patches switch 96–100% of the answers in Llama and 93–99% in Qwen (Table 6, Appendix A).

Furthermore, the vectors have norms between 0.88 and 0.94 in Llama and between 0.92 and 1.00 in Qwen, which is small compared with the token embeddings they are added to. In Llama, each patch is shorter than more than 91% of the tokens in the vocabulary and than more than 97% of the tokens that appear in the questions, whose median norm is 1.16; in Qwen, the same figures are 84% and 89%, with a median norm of 1.12. We also check that the patch does not rewrite the question into other tokens. For every token of the held-out questions, we take its patched embedding $e_i + v$ and look for the nearest token in the vocabulary, both by Euclidean distance and by cosine similarity. With the patch applied as in all our experiments, the nearest token is always the original one, for every patch in both models and for questions written in English or in either of the other two languages. This remains true when the patch is scaled up to four times its norm, even though at that scale it is longer than the typical distance between a token and its nearest neighbor (0.95–0.96 in Llama and 0.93 in Qwen), so the patch does not point toward other tokens of the vocabulary. Therefore, the model receives the same tokens but shifted along a direction, rather than different text.

4.2 The patch generalizes to new languages and to the open set

We now test whether the patches work on languages that never appear in training and on the open set.

4.2.1 Unseen input languages

Italian and Portuguese never appear in training, neither as source nor as target languages. All three patches still switch most of these questions to the target language, in both models. For Italian questions, Llama answers in the target language 89% of the time with the French patch, 91% with the Spanish one, and 97% with the German one; Qwen does so 100%, 83%, and 89% of the time. Portuguese questions switch just as reliably: 96%, 84%, and 89% in Llama, and 97%, 97%, and 99% in Qwen. Nevertheless, the switch has a cost in accuracy. Without a patch, Llama answers 72% of the Italian questions and 71% of the Portuguese ones correctly, and with a patch this drops to between 57% and 65%. In Qwen, accuracy drops from 73% (Italian) and 86% (Portuguese) to between 64% and 79%.

The patch therefore transfers the language to questions in languages it has never seen, but with a cost in accuracy. This cost is not specific to unseen languages. On English questions, the patch is as accurate as the native question (Table 1), but it is 4–16 points less accurate on questions in the other training languages (Table 6) and 4–21 points less accurate on Italian and Portuguese questions (Table 2).

Table 2: Held-out questions written in languages never used in training. Δ: accuracy of the patch minus accuracy of the native question in Table 1. *Won/Lost*: questions that the patch answers correctly and the native question does not, and vice versa.

			Target language (%)		Accuracy (%)			
Model	Patch	Question in	None	Patch	None	Patch	Δ	Won/Lost
Llama-3.2-3B	French	Italian	0	89	72	62	−14.3	9/25
		Portuguese	0	96	71	62	−13.4	5/20
	Spanish	Italian	0	91	72	58	−14.3	9/25
		Portuguese	2	84	71	63	−8.9	9/19
	German	Italian	0	97	72	57	−20.5	3/26
		Portuguese	0	89	71	65	−12.5	5/19
Qwen3-4B	French	Italian	0	100	73	71	−12.5	5/19
		Portuguese	0	97	86	79	−4.5	5/10
	Spanish	Italian	0	83	73	70	−14.3	1/17
		Portuguese	2	97	86	79	−5.4	5/11
	German	Italian	0	89	73	66	−17.9	3/23
		Portuguese	0	99	86	64	−19.6	3/25

4.2.2 Open set

On the 99 open-ended prompts, the patches yield 94–96% of answers in the target language in Llama and 89–99% in Qwen, against 98–100% for the native prompt (Table 7, Appendix B). The lowest value is the French patch in Qwen, and its failures concentrate on a single form: 10 of its 11 failures are *How do I . . . ?* prompts, answered in English with the model's usual openings (e.g., *That's a great question!*).

4.3 The patch behaves as a direction

We now test the predictions of Section 3.5 for the subtraction.

Every language patch has an inverse, all in different degrees. German is the strongest in both models, where only 3% (Llama) and 7% (Qwen) of the answers stay in German. French and Spanish keep 9% and 17% of the answers in Llama, but 62% and 52% in Qwen. In Llama, the answers that leave French and German go mostly to English, while those that leave Spanish go to French. In Qwen, the answers that leave French and Spanish go mostly to Romanian, while those that leave German go to English. Subtracting a random vector of the same norm barely changes the language, with 94–99% of the answers staying.

Table 3: Subtracting each patch from questions written in its own language. Stays: percentage of answers that remain in the language of the question; answers whose language GlotLID cannot detect are counted as leaving it. $-v$: the patch subtracted; $-r$: a random vector of the same norm subtracted, averaged over three vectors (different seeds). Destination: languages of the answers that leave with $-v$.

		Stays (%)				Accuracy (%)		
Model	Question in	None	$-v$	$-r$	Destination (%)	None	$-v$	$-r$
Llama-3.2-3B	French	99	9	99	English 84, Spanish 6, German 1	76	30	60
	Spanish	96	17	96	French 77, English 3, Italian 1, Portuguese 1	72	46	61
	German	95	3	94	English 96, Dutch 2	78	29	57
Qwen3-4B	French	100	62	97	Romanian 24, Italian 5, Spanish 4, German 3	84	76	77
	Spanish	97	52	95	Romanian 31, Portuguese 12, Catalan 3, English 2	84	45	76
	German	98	7	96	English 63, Dutch 10, Romanian 7, Spanish 6	84	27	71

The subtraction also reduces accuracy, by 26 to 49 points in Llama and by 8 to 57 points in Qwen. Part of this drop is not specific to the language. A random vector of the same norm already costs 8 to 25 points in Llama and 4 to 16 in Qwen, and for the French patch in Qwen the drop is the same as with a random vector. After manually reading the answers, we find that most of the errors come from two sources either the model replaces the question with a different one and answers it, or it asks the user for clarification without providing an answer. Llama tends to say that the question is written in a made-up language and translates it into a different question (37% of answers), while Qwen tends to say that the question contains typos and asks to rephrase it (31%). This kind of behavior emerges due to the fact that the inverse of a language is not well defined. After all, what does it mean to subtract French from a French text? Is it just keeping the content but discarding the form? The following are two very illustrative examples of this divergence.

> **User** ($-v_{\text{fr}}$): *Éclairez la formation des îles d'Hawaï*[1]
>
> **Llama:** *[...] It appears that you're trying to write in a fictional or made-up language. Am I correct? [...] it looks like you're using a combination of letters and symbols that resemble a mix of Latin and Cyrillic alphabets. Here's my attempt to translate the text: [...] "Light up the formation of the Hawaiian Islands". Please let me know if I'm on the right track or if I need to keep trying!*

Llama answers in English and it correctly identifies the content/meaning, including the nuance of the word *éclairez*, which is closer to *illuminate* than to *explain*. However, it doesn't recognize any language! It describes the question as a made-up language and even as a mix of Latin and Cyrillic alphabets, although the text is entirely in the Latin alphabet. Many of the errors follow this pattern. The model tries to understand the question, translates it as best it can, and asks the user for confirmation.

Another type of error is the misidentification of a word.

> **User** ($-v_{\text{es}}$): *Da cuenta de cómo se forman las cuevas*[2]
>
> **Qwen:** *Păstrează-ți întrebarea clară și corectă — pare că ai vrea să întrebi despre **cum se formează cuvintele** (sau „cuvav" — probabil o greșeală de tipar). Vom presupune că vrei să știi **cum se formează cuvintele** în limba română...*[3]

[1] "Shed light on the formation of the Hawaiian islands."

[2] "Explain how caves are formed"

[3] "Keep your question clear and correct — it seems you want to ask about how words are formed (or "cuvav" — probably a typo). We will assume you want to know how words are formed in the Romanian language..."

Here Qwen reads the whole question as Romanian and answers in Romanian. This may be because of the lexical closeness of some words. *Cómo* resembles the Romanian *cum* (how), and *se forman* the Romanian *se formează*. The only word that does not fit is *cuevas*, which the model maps to *cuvinte* (words). It is not fully sure of this reading, and it says so. It even quotes the word as *cuvav*, a form that exists in neither language, and attributes it to a typo. The model then answers the question it has reconstructed, explaining how words are formed in Romanian rather than how caves are formed. This example is representative of Qwen, where most of the answers that leave Spanish go to Romanian (Table 3).

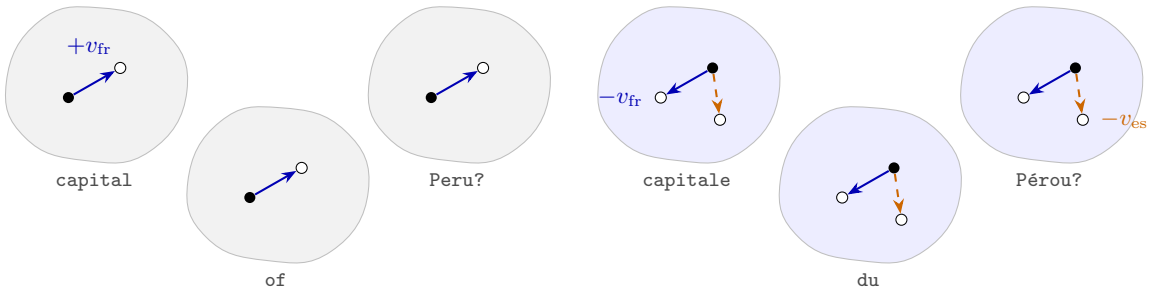
Now, we analyze the controls (Table 8, Appendix C) in three parts:

- **Another language.** Subtracting the patch of another language has a much smaller effect on the language of the answer. In Qwen, 89–100% of the answers stay in the language of the question. In Llama, this holds for most pairs (80–100%), with one exception. French and German leak into each other, since subtracting the German patch from French questions leaves 51% in French, and subtracting the French patch from German questions leaves 61%, with the remaining answers going to English.
- **Random vectors.** Random vectors with the same norm as the patch leave 92–100% of the answers in the language of the question in both models.
- **Position-bound patches.** Question-3 and Header-3 leave 92–99% and 95–100% of the answers in the language of the question, respectively, in all three languages and both models. This is not due to a smaller magnitude. The norm of Question-3 is already 1.24–1.38, against 0.88–1.00 for the shared patch, and doubling it ($\alpha = -2$) still leaves 86–100%.

In all cases, the effect is far smaller than the one obtained by subtracting the patch of the question's own language, which leaves only 3–17% of the answers in their language in Llama and 7–62% in Qwen. Together with the fact that the patch does not change the tokens of the question, these results can be pictured as in Figure 1.

The controls also differ in their cost in accuracy. Header-3 leaves accuracy unchanged, and random vectors and Question-3 reduce it moderately. Subtracting the patch of another language, in contrast, can reduce accuracy as much as subtracting the patch of the question's own language, even when the language of the answer does not change. For instance, subtracting the German patch from Spanish questions in Qwen leaves 46% accuracy with 89% of the answers in Spanish, against 45% for the Spanish patch itself.

These results support reading the language patches as directions. Subtracting a patch from a question in its own language moves the answer out of that language, strongly in Llama for all three languages and in Qwen for German, and partially in Qwen for French and Spanish. Subtracting the patch of another language has a much smaller effect, close to that of a random vector of the same norm in most pairs, except between French and German in Llama. The position-bound patches, in contrast, cannot be inverted by subtraction in any language, even at twice their magnitude.



(a) Adding v_{fr} to an English question

"represented" as French

(b) Subtracting from a French question

$-v_{fr}$: "represented" outside French
$-v_{es}$: still "represented" as French

Figure 1: Schematic of the patch in the input embedding space (not to scale). Each shaded region is the neighborhood of one token of the question.(a) The same vector v_{fr} is added to every token of an English question: each token moves by the same displacement but stays within its own neighborhood, so the text of the question is unchanged, yet the question is "represented" as French. (b) Subtracting v_{fr} from the tokens of a French question moves them in the opposite direction, out of the French "representation", whereas subtracting the Spanish direction v_{es} moves them along a different axis and leaves it unchanged.

4.4 The patch behaves as a 'translation'

In the previous section, the errors of the model on subtracted patches gave a glimpse of the mechanism behind the language change. We now take a deep dive and ask if the patch is changing the input representation (translation) or is acting as an instruction. We now show the results of experiments detailed in Section 3.5.

4.4.1 Language identification

Table 4: Language identification on held-out English questions. The patch is applied only to the question, inside a message that asks which language it is written in (Section 3.5). Each cell gives two percentages, *named* / *written*. *Named*: answers that name the language of the column. *Written*: answers written in that language, whatever language they name. *Random*: a Gaussian vector with the shape, norm, and positions of each patch. The three random controls give nearly the same result (at most 2% in any cell), so we show them in one row, with a range where they differ.

	Llama-3.2-3B			Qwen3-4B		
Question/Patch	French	Spanish	German	French	Spanish	German
English, no patch	1 / 0	2 / 0	0 / 0	0 / 0	0 / 0	0 / 0
English + Shared	76 / 1	90 / 0	85 / 0	88 / 0	97 / 0	99 / 0
English + Question-3	53 / 45	76 / 3	64 / 0	95 / 3	96 / 0	100 / 0
English + Header-3	4 / 94	2 / 99	1 / 99	83 / 100	67 / 99	56 / 100
English + Random	1 / 0	1–2 / 0	0 / 0	1 / 0	0 / 0	0 / 0
Native, no patch	100 / 0	100 / 0	96 / 0	100 / 0	100 / 1	100 / 0

Table 4 shows the results for the three formats. Random vectors of any shape have no effect, and native questions are named correctly in 96–100% of cases. We now look into the difference of the three patches:

- **Shared.** The shared patch behaves as predicted for a change in perception. The model names the language of the patch in 76–90% of cases in Llama and 88–99% in Qwen, while still writing its answer in English in almost every case (at most 1% in the target language).
- **Header-3.** This patch behaves as predicted and differs from the shared patch in two ways. First, all answers are written in the language of the patch (94–100%), as expected from an instruction to answer in that language. Second, Header-3 does not change how the model identifies the question. In Llama, for Spanish and German, the model names English but writes it in the language of the patch (e.g., *El idioma es el inglés*).[4] In French, the model exhibits a conflict between the request to name the language and the instruction of the soft prompt, and this conflict causes half of the answers to abandon the task, either answering the embedded question or translating it. In Qwen, the conflict takes another form. For Spanish and German, the model first translates the question and then names the language of its own translation, so the 56–67% of Table 4 reflects how often it translates, not a belief about the input. The only exception is French in Qwen, where the model names French without translating the question, and we cannot tell whether this reflects a change in perception or a side effect of answering in French.
- **Question-3.** Unexpectedly, Question-3 behaves like the shared patch rather than like Header-3 in five of the six cells. The model names the language of the patch in English (e.g., *Spanish*) in 64–100% of cases, and writes in English in almost every case. The exception is French in Llama, where the two patterns mix. The model answers *French*, as with the shared patch, in 33 cases, *Anglais*, as with Header-3, in 13, and *Français*, which is compatible with both, in 25, while in 27 cases it answers *English* and the patch has no effect. Its three vectors are added to the first tokens of the question, so they also modify the representation of the input, and this appears to be enough to change the perceived language, although less cleanly than the shared patch. We return to this in Discussion.

4.4.2 Conflicting instruction

Table 5 shows the results with the instruction *Please answer in English.* appended to the question. Without the instruction, all three formats yield 94–100% of answers in the target language. With it, native questions are answered in English in almost every case (96% in Llama and 100% in Qwen), so the instruction overrides the language of the question.

Now we compare the three patches:

[4] "The language is English."

Table 5: Conflicting instruction. *Please answer in English.* is appended to the question, and the patch is applied only to the question tokens. Values are the percentage of answers in the target language of the patch. Random vectors with the shape, norm, and positions of each patch give 0% in every cell.

	Llama-3.2-3B			Qwen3-4B		
Question/Patch	French	Spanish	German	French	Spanish	German
English, no patch	0	0	0	0	0	0
Native, no patch	4	4	4	0	0	0
English + Shared	23	0	7	0	0	0
English + Question-3	16	20	29	2	13	11
English + Header-3	100	99	97	98	98	97

- **Shared.** The shared patch behaves like the native question, as predicted for a change in perception. In Qwen, both are answered in English in every case. In Llama, the patched question stays in the target language in 0–7% of cases for Spanish and German, against 4% for the native question, and the difference is not significant (McNemar, $p \geq 0.12$). The largest gap is for French, where the patched question stays in French in 23% of cases against 4% for the native one ($p < 10^{-4}$). In this case the patch resists the instruction more than a genuine French question, but the instruction still prevails in most answers.
- **Header-3.** Header-3 behaves as a competing instruction. It overrides the explicit instruction in 97–100% of cases in both models, so the model keeps answering in the language of the patch even when asked to answer in English.
- **Question-3.** Question-3 lies on a middle ground, closer to the native question than to Header-3 but farther than Shared (except for French in Llama). It stays in the target language in 16–29% of cases in Llama and 2–13% in Qwen, which is more than the native question but far from Header-3.

Overall, it is clear that Header 3, the canonical version of a soft prompt, acts as an instruction, the shared patch behaves like a genuine question in the target language (to a degree), and Question-3 lies in between.

References

Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Tomas Mikolov, Wen-tau Yih, and Geoffrey Zweig. Linguistic regularities in continuous space word representations. In *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 746–751, 2013.

Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition. *Transformer Circuits Thread*, 2022. URL `https://transformer-circuits.pub/2022/toy_model/index.html`. arXiv:2209.10652.

Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. URL `https://transformer-circuits.pub/2023/monosemantic-features/index.html`.

Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, et al. Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. *Transformer Circuits Thread*, 2024. URL `https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html`.

Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. Refusal in language models is mediated by a single direction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Sarah Ball, Frauke Kreuter, and Nina Panickssery. Understanding jailbreak success: A study of latent space dynamics in large language models. *arXiv preprint arXiv:2406.09289*, 2024.

Guy Dar, Mor Geva, Ankit Gupta, and Jonathan Berant. Analyzing transformers in embedding space. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

Michael Freenor and Lauren Alvarez. Steering embedding models with geometric rotation: Mapping semantic relationships across languages and models. *arXiv preprint arXiv:2510.09790*, 2025.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

Leo Schwinn, David Dobre, Sophie Xhonneux, Gauthier Gidel, and Stephan Günnemann. Soft prompt threats: Attacking safety alignment and unlearning in open-source LLMs through the embedding space. *arXiv preprint arXiv:2402.09063*, 2024.

Yuri Kuratov, Mikhail Arkhipov, Aydar Bulatov, and Mikhail Burtsev. Cramming 1568 tokens into a single vector and back again: Exploring the limits of embedding space capacity. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

Leo Schwinn, David Dobre, Stephan Günnemann, and Gauthier Gidel. Adversarial attacks and defenses in large language models: Old and new threats. *arXiv preprint arXiv:2310.19737*, 2023.

Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Robert Belanec, Simon Ostermann, Ivan Srba, and Maria Bielikova. Task prompt vectors: Effective initialization through multi-task soft-prompt transfer. *arXiv preprint arXiv:2408.01119*, 2024.

Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. Function vectors in large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

Amir Hossein Kargaran, Ayyoob Imani, François Yvon, and Hinrich Schütze. GlotLID: Language identification for low-resource languages. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. doi:10.18653/v1/2023.findings-emnlp.410.

A Other training languages

Table 6: Held-out questions written in the other two training languages of each patch. *None*: the question without the patch, which the model answers in the language of the question. Δ: accuracy of the patch minus accuracy of the native question in Table 1. *Won/Lost*: questions that the patch answers correctly and the native question does not, and vice versa.

			Target language (%)		Accuracy (%)			
Model	Patch	Question in	None	Patch	None	Patch	Δ	Won/Lost
Llama-3.2-3B	French	Spanish	0	99	72	68	-8.0	13/22
		German	0	99	78	63	-12.5	6/20
	Spanish	French	0	96	76	66	-6.2	9/16
		German	0	97	78	66	-6.2	12/19
	German	French	0	100	76	70	-8.0	6/15
		Spanish	0	97	72	66	-11.6	7/20
Qwen3-4B	French	Spanish	0	99	84	80	-3.6	6/10
		German	0	94	84	71	-13.4	3/18
	Spanish	French	0	96	84	79	-5.4	3/9
		German	1	93	84	71	-12.5	2/16
	German	French	0	97	84	68	-16.1	3/21
		Spanish	0	99	84	71	-13.4	3/18

B Open set

Table 7: Open set, 99 open-ended prompts in English. Without a patch, the model answers in English (0% in the target language in every row). Native: the prompt written in the target language.

		Target language (%)	
Model	Patch	Native	Patch
Llama-3.2-3B	French	99	94
	Spanish	100	96
	German	99	94
Qwen3-4B	French	98	89
	Spanish	99	99
	German	99	98

C Controls for the subtraction

Table 8: Controls for the subtraction. *Stays*: percentage of answers that remain in the language of the question; answers whose language GlotLID cannot detect are counted as leaving it. *Random*: three Gaussian vectors rescaled to the norm of the shared patch of that language, one per seed, and the range covers the three seeds. *Question-3* and *Header-3*: position-bound patches of the same language. Random vectors with the shape, norm, and positions of Question-3 and Header-3 leave 94–100% of the answers in the language of the question in every cell.

		Llama-3.2-3B		Qwen3-4B	
Question in	Subtracted	Stays (%)	Acc. (%)	Stays (%)	Acc. (%)
French	None	99	76	100	84
	French patch	9	30	62	76
	Spanish patch	100	52	90	60
	German patch	51	41	100	63
	Random (3 seeds)	98–100	56–64	94–100	74–80
	Question-3	99	66	99	81
	Question-3, $\alpha = -2$	96	59	100	75
	Header-3	99	76	100	86
Spanish	None	96	72	97	84
	Spanish patch	17	46	52	45
	French patch	96	52	93	80
	German patch	80	46	89	46
	Random (3 seeds)	93–99	59–64	93–96	71–79
	Question-3	95	70	96	84
	Question-3, $\alpha = -2$	86	70	87	84
	Header-3	96	78	97	84
German	None	95	78	98	84
	German patch	3	29	7	27
	French patch	61	39	97	77
	Spanish patch	100	63	89	57
	Random (3 seeds)	92–96	53–61	95–97	68–73
	Question-3	92	70	97	81
	Question-3, $\alpha = -2$	87	60	96	69
	Header-3	95	77	99	84